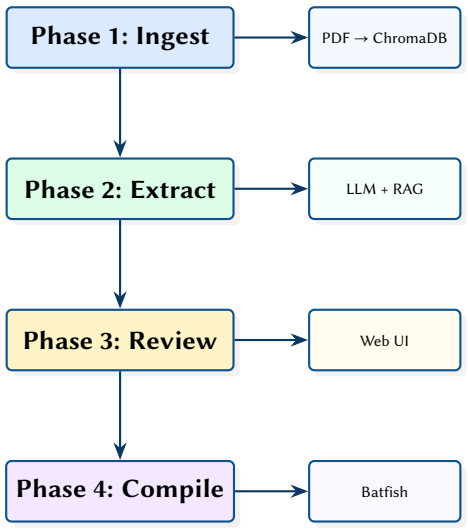
LLM-Powered Network Configuration Parsing and Cross-Vendor Compilation

RAG-Based Extraction, Human-in-the-Loop Review, and Semantic Validation

Simon Knight

March 2026 • Version 1.0.0

Last updated: March 11, 2026



Abstract. This report presents a network automation framework that decouples configuration from vendor-specific syntax using LLM-powered Retrieval-Augmented Generation. The system operates in four phases: vendor documentation ingestion into a ChromaDB vector store, LLM-powered topology extraction with confidence scoring and evidence tracking, human-in-the-loop web-based review with citation enforcement, and cross-vendor configuration compilation with Batfish semantic validation. The vendor-neutral intermediate representation models network relationships (interfaces, VLANs, OSPF, BGP) rather than device syntax, enabling the same topology to compile to Cisco IOS and Arista EOS output with full evidence traceability.

Version	Date	Changes
1.0.0	March 2026	Initial report (all four phases complete)

Contents

List of Design Decisions & Rules

1 Introduction and Motivation

Network configuration is inherently vendor-specific: the same logical intent ("interface Ethernet1 has IP 10.0.0.1/30 in VLAN 100") is expressed differently across Cisco IOS, Arista EOS, Juniper JunOS, and others. Traditional parsers use hand-written regular expressions or TextFSM templates, which are brittle, vendor-locked, and require constant maintenance as CLI syntax evolves.

This framework takes a fundamentally different approach: it uses Large Language Model (LLM)-powered Retrieval-Augmented Generation (RAG) [1] to understand vendor documentation and extract network intent from configurations, producing a vendor-neutral Intermediate Representation (IR) that can be compiled to any target vendor.

: Evidence-Backed Extraction

Every extracted field carries a confidence score (0.0–1.0) and evidence citations linking the value to source configuration lines and documentation chunks. Low-confidence extractions are automatically flagged for human review.

2 Phase 1: Knowledge Base Ingestion

Vendor documentation (PDF manuals, CLI references) is ingested through a layout-aware PDF parser that preserves section structure and code blocks. Parsed chunks are embedded and stored in ChromaDB [2] with deterministic chunk IDs for reproducible citations.

```
class DocumentIngester:
    def ingest_pdf(self, path: Path, vendor: str) -> int:
        chunks = self.parser.extract_chunks(path)
        for chunk in chunks:
            chunk_id = deterministic_id(vendor, path.name, chunk.page, chunk.index)
            self.collection.add(
                ids=[chunk_id],
                documents=[chunk.text],
                metadatas=[{"vendor": vendor, "page": chunk.page}],
            )
        return len(chunks)
```

3 Phase 2: Topology Extraction

Insight: *Deterministic chunk IDs ensure that re-ingesting the same document produces identical identifiers, preserving evidence citations across extractions without invalidating the overlay database.*

The extraction engine processes vendor configurations through an LLM with RAG context from the knowledge base. For each configuration section, relevant documentation chunks are retrieved from ChromaDB and included in the prompt, giving the LLM vendor-specific context for accurate parsing.

The output is a vendor-neutral IR modelling:

- **Interfaces** — name, IP addresses, VLANs, status
- **Routing** — Open Shortest Path First (OSPF) areas, Border Gateway Protocol (BGP) peers, static routes
- **Services** — DHCP, NTP, DNS, ACLs

Each extracted field includes:

```
@dataclass
class ExtractedField:
    value: Any
    confidence: float          # 0.0 - 1.0
    config_evidence: list[str] # Source config line numbers
    doc_evidence: list[str]    # ChromaDB chunk IDs
```

Design Decision 1: Hallucination Protection

Fields extracted with confidence below the configurable threshold (default 0.6) are automatically flagged for Human-in-the-Loop (HITL) review. The evidence chain (config line → doc chunk → extracted value) enables reviewers to verify the LLM's reasoning rather than trusting the output blindly.

4 Phase 3: Human-in-the-Loop Review

A web-based review interface presents flagged extractions with their evidence chains. Reviewers can:

- **Approve** — Accept the extraction as-is (confidence set to 1.0).
- **Edit** — Modify the value with mandatory evidence citation.
- **Reject** — Mark as incorrect; excluded from compilation.

Approved edits are stored as overlays that persist across re-extractions. When the extraction pipeline runs again (e.g. after documentation updates), approved overlays are applied automatically, preserving human corrections.

: Evidence Enforcement

All manual edits must cite either a configuration line number or a documentation chunk ID. This prevents unsupported assertions from entering the IR and maintains the end-to-end evidence chain.

5 Phase 4: Cross-Vendor Compilation

The compiler transforms the vendor-neutral IR into target vendor configurations using Jinja2 templates. Currently supported targets are Cisco IOS and Arista EOS.

Generated configurations include evidence comments tracing each line back to the IR field and its original source:

```
! Source: interface Ethernet1 (confidence: 0.95)
! Evidence: config line 42, doc chunk cisco-ios-ref:p234:3
interface Ethernet1
 ip address 10.0.0.1 255.255.255.252
 no shutdown
```

5.1 Semantic Validation

Compiled configurations are validated using Batfish [3], which performs semantic analysis to verify:

- Reachability between expected endpoint pairs
- Route table correctness (expected prefixes present)
- ACL behaviour matches intent
- No unintended route leaking between VRFs

6 Design Summary

Insight: *Batfish validation catches semantic errors that syntactic template validation misses—for example, a correctly formatted OSPF configuration that advertises the wrong network due to a wildcard mask error.*

Table 1. Key design decisions.

Decision	Rationale
RAG over fine-tuning	Vendor docs change frequently; RAG adapts without retraining
Confidence scoring	Quantifies extraction reliability; enables automatic HITL routing
Deterministic chunk IDs	Stable citations across re-ingestion; overlays survive doc updates
Overlay persistence	Human corrections survive re-extraction without manual re-review
Batfish validation	Semantic verification catches errors invisible to syntactic checks

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," *NeurIPS*, 2020.

[2] Chroma, *ChromaDB: The open-source embedding database*, 2024. [Online]. Available: https://www.trychroma.com

[3] Intentionet, *Batfish: Network configuration analysis*, 2024. [Online]. Available: https://www.batfish.org